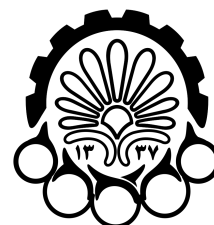


دانشگاه صنعتی امیرکبیر
دانشکده مهندسی کامپیوتر



مبانی رایانش ابری
نیمسال دوم ۱۴۰۴-۱۴۰۵



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

فاز اول پروژه پایانی

تحلیل لاگ جام جهانی با Nginx
و MapReduce

دکتر جوادی

استاد درس

• مهرداد شیخ عباسی

• محمد هادی ستاک

تدریس‌یاران مربوطه

جمعه ۲ مرداد ۱۴۰۵، ساعت ۲۳:۵۹

زمان تحویل

مورد تحویلی

نام فایل زیپ نهایی باید مطابق قالب CC_Project_Phase1_StudentID1_StudentID2.zip باشد. در این نام‌گذاری، StudentID1 و StudentID2 باید با شماره دانشجویی دو عضو گروه جایگزین شوند. برای مثال، اگر شماره دانشجویی دو عضو گروه 40123456 و 40234567 باشد، نام فایل زیپ باید به صورت CC_Project_Phase1_40123456_40234567.zip انتخاب شود.

لطفاً مراحل این دستورکار را به ترتیب انجام دهید و در صورت نیاز، خروجی‌های خواسته شده را ثبت کنید.

۱. هدف فاز اول

هدف فاز اول پروژه، ساخت یک سامانه ساده برای تولید و تحلیل لاگ است. در این فاز، دانشجوها باید چند سرویس ساده Python را پشت Nginx اجرا کنند، با یک برنامه تولیدکننده ترافیک به آن‌ها درخواست بفرستند، لاگ‌های تولیدشده را با Hadoop Streaming و کدهای Python پردازش کنند و در پایان چند خروجی تحلیلی مشخص بسازند.

در این فاز، تمرکز اصلی روی مفاهیم زیر است:

- اجرای چند سرویس ساده با Docker و Docker Compose؛
- استفاده از Nginx به عنوان API Gateway یا reverse proxy؛
- تولید ترافیک و تولید لاگ قابل تحلیل؛
- طراحی schema مشخص برای لاگ‌ها؛
- پردازش batch لاگ‌ها با MapReduce؛
- تولید خروجی‌های تحلیلی عمومی و سناریومحور؛
- انجام تحلیل real-time با Spark Structured Streaming به صورت امتیازی.

۲. سناریوی پروژه

سناریوی این فاز، تحلیل لاگ یک سامانه ساده اطلاعات جام جهانی است. در این سامانه، کاربرها از کشورهای مختلف به سرویس‌های مختلف درخواست می‌فرستند و درباره بازی‌ها، تیم‌ها و ورزشگاه‌ها اطلاعات می‌گیرند. سامانه شامل سه سرویس ساده است:

- match-service: دریافت تاریخ و برگرداندن بازی‌های آن روز؛
- team-service: دریافت نام تیم و برگرداندن اطلاعات ساده تیم؛
- stadium-service: دریافت نام ورزشگاه یا شهر و برگرداندن اطلاعات مربوط به آن.

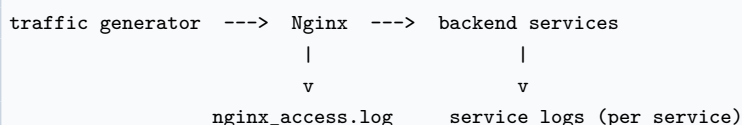
نکته

کد پایه‌ی سه سرویس match-service، team-service و stadium-service در اختیار دانشجوها قرار می‌گیرد. این کد پایه فقط برای داشتن endpointهای اولیه است و بخش‌های مربوط به تولید لاگ ساختاریافته و ثبت فیلدهای مربوط به موجودیت با TODO مشخص می‌شود. مقدار headerهای لازم در کد پایه خوانده شده و باید در رکورد لاگ استفاده شود. تکمیل همین بخش‌های ناقص، نوشتن Dockerfileها، تنظیم Nginx، نوشتن traffic generator و پیاده‌سازی MapReduce بر عهده‌ی دانشجوهاست.

۳. معماری مورد انتظار

تمام درخواست‌ها باید ابتدا به Nginx ارسال شوند و سپس Nginx آن‌ها را به سرویس مناسب منتقل کند. در اجرای نهایی، traffic generator نباید مستقیماً به سرویس‌های backend درخواست بفرستد.

مسیر صحیح درخواست‌ها به شکل زیر است:



در این معماری دو دسته لاگ تولید می‌شود: Nginx gateway log برای تحلیل‌های عمومی و لاگ اختصاصی هر سرویس backend برای تحلیل‌های مخصوص سرویس‌ها. این جداسازی در بخش «فرمت لاگ‌ها» با جزئیات توضیح داده می‌شود.

ساختار کلی اجزای پروژه باید شامل موارد زیر باشد:

- سه سرویس Python ساده که هر کدام لاگ ساختاریافته خود را می‌نویسند؛
- یک Nginx به عنوان reverse proxy و تولیدکننده nginx_access.log؛
- یک برنامه Python برای تولید ترافیک (فقط نقش کاربر آزمایشی)؛
- Nginx gateway log برای تحلیل‌های عمومی؛
- لاگ‌های اختصاصی سرویس‌ها برای تحلیل‌های مخصوص سرویس‌ها؛
- کدهای MapReduce با Hadoop Streaming؛
- خروجی‌های نهایی تحلیل.

۴. سرویس‌های backend

هر گروه باید سه سرویس ساده Python داشته باشد. استفاده از FastAPI یا Flask آزاد است، اما همه سرویس‌ها باید قابل اجرا داخل Docker باشند.

۴.۱ سرویس match-service

این سرویس با گرفتن یک تاریخ، بازی‌های آن روز را برمی‌گرداند.
نمونه endpoint:

```
GET /api/matches?date=2026-06-25
```

رفتارهای مورد انتظار:

- برای تاریخ‌های معتبر، پاسخ موفق برگرداند؛
- برای تاریخ نامعتبر یا بدون بازی، پاسخ مناسب با status code مشخص برگرداند؛
- در برخی درخواست‌ها بتواند پاسخ کندتر تولید کند تا تحلیل response time معنی‌دار شود.

۴.۲. سرویس team-service

این سرویس با گرفتن نام یک تیم، اطلاعات ساده‌ای درباره آن تیم برمی‌گرداند.
نمونه endpoint:

```
GET /api/teams?name=Argentina
```

رفتارهای مورد انتظار:

- برای تیم‌های موجود، پاسخ موفق برگرداند؛
- برای تیم‌های ناموجود، پاسخ خطای مناسب برگرداند؛
- داده‌ها به اندازه‌ای متنوع باشند که بتوان محبوبیت تیم‌ها را از روی تعداد درخواست‌ها تحلیل کرد.

۴.۳. سرویس stadium-service

این سرویس با گرفتن نام ورزشگاه یا شهر، اطلاعات ساده مربوط به آن را برمی‌گرداند.
نمونه endpoint:

```
GET /api/stadiums?name=New York New Jersey Stadium
GET /api/stadiums?city=New York New Jersey
```

رفتارهای مورد انتظار:

- برای ورزشگاه‌ها یا شهرهای معتبر، پاسخ موفق برگرداند؛
- برای ورودی نامعتبر، پاسخ خطای مناسب برگرداند؛
- امکان تحلیل پر جست‌وجوترین ورزشگاه یا شهر وجود داشته باشد.

۴.۴. داکرایز کردن سرویس‌ها

برای هر سه سرویس باید یک Dockerfile مستقل نوشته شود تا سرویس داخل کانتینر اجرا شود. انتظار می‌رود هر سرویس حداقل این موارد را داشته باشد:

- فایل requirements.txt برای وابستگی‌های Python؛
- فایل Dockerfile مستقل؛
- اجرای سرویس روی 0.0.0.0 تا از داخل شبکه Docker Compose قابل دسترسی باشد؛
- پورت داخلی مشخص برای هر سرویس؛
- عدم وابستگی به مسیرهای مطلق روی سیستم شخصی دانشجو؛
- نوشتن یک لاگ ساختاریافته اختصاصی به صورت JSON Lines در مسیر data/service_logs/؛
- استفاده از مقدارهای X-Request-ID، X-Client-Country و X-Scenario که Nginx به سرویس منتقل می‌کند، در رکورد لاگ ساختاریافته.

نکته

کد پایه‌ی سه سرویس match-service، team-service و stadium-service در اختیار دانشجویان قرار می‌گیرد، اما بخش لاگ‌گذاری تحلیلی آماده نیست و باید توسط گروه تکمیل شود. در این پروژه دو دسته لاگ داریم: nginx_access.log که Nginx تولید می‌کند و برای تحلیل‌های عمومی و gateway-level به کار می‌رود؛ و لاگ اختصاصی هر سرویس backend که حاوی اطلاعات مربوط به موجودیت (entity_type و entity_value) است و برای تحلیل‌های مخصوص سرویس‌ها استفاده می‌شود. traffic generator فقط نقش کاربر آزمایشی را دارد و لاگ آن مبنای تحلیل نیست.

۵. تنظیم Nginx

Nginx باید جلوی سه سرویس قرار بگیرد و بر اساس مسیر درخواست، آن را به سرویس مناسب منتقل کند. برای مثال، در پروژه واقعی شما مسیرهای مربوط به بازی‌ها، تیم‌ها و ورزشگاه‌ها باید به سرویس متناظر ارسال شوند؛ اما در این بخش، برای هدف آموزشی، یک مثال فرضی و ساده توضیح داده می‌شود تا ساختار کانفیگ مشخص شود بدون اینکه جواب مستقیم پروژه داده شود.

Nginx gateway log باید از Nginx access log به دست بیاید؛ زیرا همه درخواست‌ها از Nginx عبور می‌کنند. این لاگ برای تحلیل‌های عمومی استفاده می‌شود و تحلیل‌های مخصوص سرویس‌ها از لاگ سرویس‌ها به دست می‌آیند.

توجه

در اجرای نهایی، اگر traffic generator مستقیماً به سرویس‌های backend درخواست بفرستد و Nginx دور زده شود، اجرای پروژه ناقص محسوب می‌شود.

۵.۱. نمونه آموزشی کانفیگ و توضیح دستوره‌های Nginx

نمونه زیر مربوط به یک پروژه فرضی است که دو سرویس service-a و service-b دارد. دانشجویها باید همین ایده را برای مسیرها و سرویس‌های پروژه خودشان پیاده‌سازی کنند.

```
events {}

http {
    upstream service_a_upstream {
        server service-a:8000;
    }

    upstream service_b_upstream {
        server service-b:8000;
    }

    log_format json_logs escape=json
        '{"timestamp": "$time_iso8601",'
        '"request_id": "$http_x_request_id",'
        '"client_ip": "$remote_addr",'
        '"client_country": "$http_x_client_country",'
        '"scenario": "$http_x_scenario",'
        '"method": "$request_method",'
        '"path": "$request_uri",'
        '"service": "$target_service",'
        '"status_code": $status,'
        '"request_time_sec": "$request_time",'
        '"user_agent": "$http_user_agent"}';

    access_log /var/log/nginx/nginx_access.log json_logs;

    server {
        listen 80;

        # default value so the "service" log field is never empty
        set $target_service "unknown";

        location /api/a {
            set $target_service service-a;
            proxy_pass http://service_a_upstream;
            proxy_set_header X-Request-ID $http_x_request_id;
            proxy_set_header X-Client-Country $http_x_client_country;
            proxy_set_header X-Scenario $http_x_scenario;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }

        location /api/b {
            set $target_service service-b;
            proxy_pass http://service_b_upstream;
            proxy_set_header X-Request-ID $http_x_request_id;
            proxy_set_header X-Client-Country $http_x_client_country;
            proxy_set_header X-Scenario $http_x_scenario;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }
    }
}
```

```
}  
}
```

توضیح دستورهای مهم:

- `events`: بخش پایه Nginx برای تنظیم اتصال هاست و در این پروژه می تواند حداقلی باشد.
- `http`: همه تنظیمات مربوط به درخواست های HTTP داخل این بخش قرار می گیرد.
- `upstream`: برای تعریف مقصدهای داخلی استفاده می شود. وقتی سرویس ها با Docker Compose اجرا می شوند، نام هایی مثل `service-a` همان نام سرویس در `docker-compose.yml` هستند.
- `server service-a:8000`: یعنی درخواست ها به سرویس `service-a` روی پورت داخلی 8000 فرستاده شوند.
- `log_format`: شکل لاگ را مشخص می کند. در این پروژه، لاگ باید ساختاریافته و قابل پردازش با MapReduce باشد.
- `escape=json`: کمک می کند مقادارهای داخل لاگ برای خروجی JSON بهتر escape شوند.
- `access_log`: مسیر ذخیره لاگ و فرمت آن را مشخص می کند. این فایل، ورودی تحلیل های عمومی MapReduce است (تحلیل های مخصوص سرویس ها از لاگ سرویس ها می آیند).
- `listen 80` و `server`: مشخص می کند Nginx داخل کانتینر روی پورت 80 گوش می دهد. نگاشت این پورت به هاست باید در `docker-compose.yml` انجام شود.
- `location`: مسیرهای مختلف را از هم جدا می کند و هر مسیر را به سرویس مناسب می فرستد.
- `set $target_service`: نام سرویس مقصد را داخل یک متغیر قرار می دهد تا همان نام در لاگ ذخیره شود. بهتر است یک مقدار پیش فرض (مثلاً `unknown`) در سطح `server` تعریف کنید تا برای درخواست هایی که به هیچ `location` نمی خورند، این فیلد خالی نماند.
- `proxy_pass`: درخواست را به `upstream` مناسب منتقل می کند.
- `proxy_set_header`: headerهای مهم مثل کشور، سناریو و `request_id` را به سرویس مقصد منتقل می کند.
- `$request_time`: مدت زمان پردازش درخواست در Nginx را برحسب ثانیه نشان می دهد. در Job 1 می توان آن را به `response_time_ms` تبدیل کرد.

نکته

نمونه بالا یک `nginx.conf` کامل است (شامل بلوک های `events` و `http`). بنابراین در `docker-compose.yml` باید این فایل را به مسیر `/etc/nginx/nginx.conf` متصل (mount) کنید، نه داخل `conf.d/`؛ در غیر این صورت Nginx اجرا نمی شود. همچنین پورت کانتینر را به هاست نگاشت

کنید، مثلاً `ports: ["8080:80"]`.

۵.۲. مسیر اجرا و تست Nginx

بعد از تکمیل `Dockerfile` ها، `docker-compose.yml` و `nginx.conf`، سرویس ها را اجرا کنید:

```
docker compose up --build -d
```

وضعیت کانتینرها را بررسی کنید:

```
docker compose ps
```

درستی کانفیگ Nginx را تست کنید:

```
docker compose exec nginx nginx -t
```

اگر کانفیگ را تغییر دادید و نخواستید کل سیستم را `restart` کنید:

```
docker compose exec nginx nginx -s reload
```

برای تست دستی یک درخواست، حتماً به Nginx درخواست بفرستید، نه مستقیماً به `backend`. مسیر زیر فقط نمونه است و باید با یکی از مسیرهای پروژه خودتان جایگزین شود:

```
curl -H "X-Request-ID: test-001" \\\n  -H "X-Client-Country: Iran" \\\n  -H "X-Scenario: manual-test" \\\n  "http://localhost:8080/<PROJECT_ENDPOINT>"
```

برای مشاهده لاگ تولیدشده:

```
docker compose exec nginx sh -c "tail -n 5 /var/log/nginx/nginx_access.log"
```

اگر مسیر لاگ Nginx را با `volume` به سیستم میزبان متصل کرده‌اید، می‌توانید همان فایل را از روی هاست نیز مشاهده کنید:

```
tail -n 5 data/nginx/nginx_access.log
```

نتیجه مورد انتظار

بعد از اجرای `curl` باید یک خط جدید در `nginx_access.log` دیده شود. اگر `request_id`، `client_country`، `service` و `path` در لاگ نیامده باشند، کانفیگ لاگ کامل نیست.

توجه

برای شروع یک اجرای تمیز، فایل لاگ زنده‌ای را که Nginx در حال نوشتن آن است حذف نکنید. اگر لاگ را با `volume` به هاست متصل کرده‌اید، حذف مستقیم فایل ممکن است باعث شود Nginx همچنان در یک

فایل حذف شده بنویسد. برای پاک‌سازی امن، Nginx را restart کنید یا فایل را به صورت امن truncate کنید.

۶. تولید ترافیک

دانشجوها باید یک برنامه Python برای تولید ترافیک بنویسند. این برنامه نقش کاربرهای واقعی را بازی می‌کند و باید به آدرس Nginx درخواست بفرستد.

این برنامه باید هنگام ارسال هر درخواست، دست‌کم headerهای زیر را تنظیم کند:

```
X-Request-ID: req_000001
X-Client-Country: Iran
X-Scenario: normal
```

فیلد X-Client-Country برای تحلیل‌هایی مانند «هر کشور بیشتر طرفدار کدام تیم است» استفاده می‌شود. در این پروژه لازم نیست کشور از روی IP تشخیص داده شود.

توجه

traffic generator نباید به عنوان منبع اصلی تحلیل MapReduce استفاده شود. این برنامه فقط برای تولید درخواست و تست سیستم است. اگر دانشجو برای debug یک فایل کمکی مثل generator_trace.csv تولید کند، آن فایل نباید جایگزین nginx_access.log شود و نباید مبنای خروجی‌های اصلی MapReduce باشد.

۶.۱ سناریوهای اجباری ترافیک

traffic generator باید سناریوهای زیر را تولید کند:

- درخواست‌های عادی به هر سه سرویس؛
- درخواست به تیم‌ها، تاریخ‌ها یا ورزشگاه‌های مختلف؛
- درخواست با ورودی نامعتبر که خطای 4xx تولید کند؛
- درخواست‌هایی که باعث خطای 5xx شوند؛
- ترافیک نامتوازن، مثلاً درخواست بیشتر برای یک تیم، یک روز یا یک ورزشگاه خاص؛
- درخواست از چند کشور مختلف، با مقدارهای متفاوت در X-Client-Country.

۶.۲ حجم داده

برای تست اولیه، اجرای پروژه با تعداد کم درخواست مجاز است؛ اما خروجی نهایی باید با حجم کافی تولید شود:

```
Debug run: at least 1,000 requests
Final run: at least 100,000 requests
```

در اجرای نهایی، داده‌ها باید به اندازه‌ای متنوع باشند که تفاوت بین سرویس‌ها، endpointها، کشورها، تیم‌ها، تاریخ‌ها و ورزشگاه‌ها در خروجی‌ها قابل مشاهده باشد.

۷. فرمت لاگ‌ها

در این پروژه دو دسته لاگ برای تحلیل داریم. Nginx gateway log و nginx_access.log که همه درخواست‌های عبوری از Nginx را ثبت می‌کند و مبنای تحلیل‌های عمومی است؛ و لاگ‌های اختصاصی هر سرویس backend در مسیر data/service_logs/ که مبنای تحلیل‌های مخصوص سرویس‌ها هستند.

۷.۱. فرمت nginx_access.log

پیشنهاد می‌شود لاگ Nginx به صورت JSON Lines ذخیره شود؛ یعنی هر خط یک JSON مستقل باشد. فیلدهای اجباری:

```
timestamp
request_id
client_ip
client_country
scenario
method
path
service
status_code
request_time_sec
user_agent
```

نمونه خط لاگ:

```
{
  "timestamp": "2026-06-25T12:00:01",
  "request_id": "req_000001",
  "client_ip": "172.18.0.1",
  "client_country": "Iran",
  "scenario": "normal",
  "method": "GET",
  "path": "/api/teams?name=Argentina",
  "service": "team-service",
  "status_code": 200,
  "request_time_sec": "0.043",
  "user_agent": "traffic-generator"
}
```

۷.۲. فرمت لاگ سرویس‌های backend

هر سرویس backend باید برای هر درخواست یک خط لاگ ساختاریافته به صورت JSON Lines بنویسد (هر خط یک JSON مستقل).

فیلدهای مشترک و اجباری لاگ هر سرویس:

```
timestamp      # event time
request_id     # from X-Request-ID header
client_country # from X-Client-Country header
service        # e.g. team-service
endpoint       # path without query, e.g. /api/teams
entity_type    # team | match_day | stadium | city
entity_value   # e.g. Argentina
status_code    # response status code
processing_time_ms # in-service processing time in milliseconds
event_type     # e.g. team_lookup
```

توضیح کوتاه فیلدها: timestamp زمان رویداد؛ request_id همان شناسه‌ای که از هدر X-Request-ID می‌آید (برای ردگیری یک درخواست در هر دو لاگ)؛ client_country از هدر X-Client-Country؛ service؛ endpoint نام سرویس و مسیر بدون query؛ entity_value/entity_type موجودیت مربوط به درخواست؛ status_code کد وضعیت پاسخ؛ processing_time_ms زمان پردازش داخل سرویس؛ و event_type نوع رویداد.

سرویس باید هدرهای X-Request-ID، X-Client-Country و X-Scenario را که Nginx منتقل می‌کند بخواند و در لاگ خود استفاده کند.

۷.۳. فیلدهای هر سرویس

match-service: مقدار entity_type برابر match_day است و entity_value همان تاریخ درخواستی است. نمونه درخواست:

```
GET /api/matches?date=2026-06-25
```

```
{
  "timestamp": "...",
  "request_id": "req_000001",
  "client_country": "Iran",
  "service": "match-service",
  "endpoint": "/api/matches",
  "entity_type": "match_day",
  "entity_value": "2026-06-25",
  "status_code": 200,
  "processing_time_ms": 42,
  "event_type": "match_lookup"
}
```

team-service: مقدار entity_type برابر team است و entity_value نام تیم است. نمونه درخواست:

```
GET /api/teams?name=Argentina
```

```
{
  "timestamp": "...",
  "request_id": "req_000002",
  "client_country": "Germany",
  "service": "team-service",
  "endpoint": "/api/teams",
  "entity_type": "team",
  "entity_value": "Argentina",
  "status_code": 200,
  "processing_time_ms": 31,
  "event_type": "team_lookup"
}
```

stadium-service: مقدار entity_type برابر stadium یا city است. نمونه درخواست:

```
GET /api/stadiums?name=New York New Jersey Stadium
GET /api/stadiums?city=New York New Jersey
```

```
{
  "timestamp": "...",
  "request_id": "req_000003",
  "client_country": "Brazil",
  "service": "stadium-service",
  "endpoint": "/api/stadiums",
  "entity_type": "stadium",
  "entity_value": "New York New Jersey Stadium",
  "status_code": 200,
  "processing_time_ms": 27,
  "event_type": "stadium_lookup"
}
```

۸. پردازش با Hadoop Streaming

پردازش batch در این فاز باید با Hadoop Streaming انجام شود. دانشجویان می‌توانند mapper و reducerها را با Python بنویسند و نیازی به پیاده‌سازی MapReduce با Java نیست.

توجه

استفاده از Pandas یا یک اسکریپت ساده Python به جای Hadoop Streaming برای بخش اصلی MapReduce کافی نیست. خروجی‌ها باید از اجرای mapper و reducer در جریان Hadoop Streaming تولید شوند.

۸.۱ اجرای کلاستر Hadoop با Docker Compose

برای اجرای بخش Hadoop Streaming، یک فایل docker-compose.yml نمونه برای محیط Hadoop در اختیار دانشجویان قرار می‌گیرد. این فایل فقط کلاستر ساده‌ی Hadoop را بالا می‌آورد و جواب پروژه، کد MapReduce، فایل Nginx، traffic generator یا خروجی‌های نهایی را شامل نمی‌شود. برای بالا آوردن کلاستر، در پوشه‌ای که فایل docker-compose.yml نمونه قرار دارد دستور زیر را اجرا کنید:

```
docker compose up -d
```

برای بررسی کانتینرهای ساخته‌شده:

```
docker compose ps
```

این compose نمونه معمولاً دو کانتینر اصلی بالا می‌آورد:

- namenode: مدیریت ساختار HDFS و نقطه‌ی اصلی اجرای دستورهای Hadoop؛
- datanode: نگهداری واقعی بلوک‌های داده در HDFS.

بعد از بالا آمدن کلاستر، می‌توانید رابط‌های گرافیکی Hadoop را در مرورگر ببینید. در تنظیم رایج، NameNode روی آدرس <http://127.0.0.1:9870> و DataNode UI روی آدرس <http://127.0.0.1:9864> در دسترس است. اگر در فایل docker-compose.yml پورت‌ها متفاوت تنظیم شده باشند، همان پورت‌های فایل compose ملاک هستند.

برای ورود به کانتینر namenode:

```
docker compose exec namenode bash
```

برای متوقف کردن کلاستر نیز می‌توانید از دستور زیر استفاده کنید:

```
docker compose down
```

۸.۲. قرار دادن فایل ورودی روی HDFS

فایل‌های پروژه روی سیستم شما قرار دارند، اما MapReduce باید بتواند آن‌ها را از داخل محیط Hadoop بخواند. اگر در فایل `docker-compose.yml` پوشه‌ی پروژه با `volume mount` داخل کانتینر دیده شود، فایل‌های محلی مثلاً از مسیر `/project/data` داخل کانتینر قابل دسترسی هستند. این روش برای نگه داشتن کدها، لاگ‌ها و خروجی‌های پروژه روی سیستم میزبان مناسب است.

با این حال، برای اجرای MapReduce روی HDFS، باید فایل ورودی را با دستور `hdfs dfs -put` وارد HDFS کنید. این دستور معمولاً داخل کانتینر `namenode` اجرا می‌شود. `NameNode` مسیر فایل را مدیریت می‌کند و داده‌ها در نهایت روی `DataNode` ذخیره می‌شوند.

نمونه‌ی کلی:

```
docker compose exec namenode bash
```

```
hdfs dfs -mkdir -p /input
hdfs dfs -put -f /project/data/nginx/nginx_access.log /input/nginx_access.log
hdfs dfs -ls /input
```

اگر می‌خواهید لاگ‌های سرویس‌ها را نیز برای Job 1 وارد HDFS کنید، می‌توانید همین الگو را برای مسیرهای زیر تکرار کنید:

```
hdfs dfs -mkdir -p /input/service_logs
hdfs dfs -put -f /project/data/service_logs/match_service.log /input/service_logs/match_service.log
hdfs dfs -put -f /project/data/service_logs/team_service.log /input/service_logs/team_service.log
hdfs dfs -put -f /project/data/service_logs/stadium_service.log /input/service_logs/stadium_service.log
hdfs dfs -ls /input/service_logs
```

نکته

برای جابه‌جایی فایل بین سیستم میزبان و کانتینرها، روش اصلی و قابل تکرار `volume mount` است. یعنی پوشه‌هایی مثل `data/`، `mapreduce/`، `scripts/` و `outputs/` روی سیستم میزبان، داخل کانتینر نیز روی مسیری مثل `/project` دیده شوند. سپس برای اجرای واقعی روی HDFS، فایل‌های ورودی با `hdfs dfs -put` وارد HDFS می‌شوند و خروجی‌ها بعد از اجرا با `hdfs dfs -cat` یا `hdfs dfs -getmerge` به پوشه‌ی `outputs/` برگردانده می‌شوند. استفاده از `docker cp` به عنوان روش اصلی پیشنهاد نمی‌شود.

۸.۳. تست یک MapReduce ساده با Python

Hadoop معمولاً برنامه‌های MapReduce را با Java اجرا می‌کند، اما Hadoop Streaming اجازه می‌دهد برنامه‌های خارجی مثل اسکریپت‌های Python به عنوان `mapper` و `reducer` استفاده شوند. در این روش،

Hadoop داده را از stdin به mapper می‌دهد و خروجی mapper را از stdout می‌خواند؛ برای reducer نیز همین روند انجام می‌شود.

برای اطمینان از اینکه محیط درست کار می‌کند، ابتدا یک تست ساده انجام دهید. فرض کنید فایل‌های زیر را در پوشه‌ی mapreduce/test/ نوشته‌اید.

نکته

توضیحات موجود در این بخش (۸.۳) فقط جنبه آموزشی دارند و لازم نیست که دانشجویان آن را اجرا کنند. هدف این است که دانشجویان با نحوه‌ی اجرای Hadoop Streaming و ساختار mapper و reducer آشنا شوند. در پروژه‌ی اصلی، دانشجویان باید mapper و reducerهای خودشان را برای تحلیل‌های مورد نیاز بنویسند.

mapper ساده که برای هر خط یک مقدار 1 تولید می‌کند:

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    line = line.strip()
    if line:
        print("total\t1")
```

reducer ساده که تعداد خط‌ها را جمع می‌کند:

```
#!/usr/bin/env python3
import sys

count = 0
for line in sys.stdin:
    key, value = line.strip().split("\t")
    count += int(value)

print("total\t%d" % count)
```

قبل از اجرای Hadoop، می‌توانید همین دو فایل را به صورت محلی از داخل کانتینر تست کنید:

```
cd /project
cat data/nginx/nginx_access.log | \
    python3 mapreduce/test/mapper_line_count.py | \
    sort | \
    python3 mapreduce/test/reducer_sum.py
```

این تست فقط برای debug است و جایگزین اجرای Hadoop Streaming نیست.

برای اجرای همین تست با Hadoop Streaming، ابتدا مطمئن شوید فایل ورودی در HDFS قرار دارد و خروجی قبلی وجود ندارد:

```
hdfs dfs -mkdir -p /input
hdfs dfs -put -f /project/data/nginx/nginx_access.log /input/nginx_access.log
hdfs dfs -rm -r -f /output/test_line_count
```

سپس job را اجرا کنید:

```
hadoop jar /opt/hadoop-3.2.1/share/hadoop/tools/lib/hadoop-streaming-3.2.1.jar \
-files /project/mapreduce/test/mapper_line_count.py,/project/mapreduce/test/reducer_sum.py \
-mapper "python3 mapper_line_count.py" \
-reducer "python3 reducer_sum.py" \
-input /input/nginx_access.log \
-output /output/test_line_count
```

برای دیدن خروجی:

```
hdfs dfs -cat /output/test_line_count/part-*
```

توجه

مسیر output- در Hadoop نباید از قبل وجود داشته باشد. قبل از اجرای دوباره‌ی هر job، مسیر خروجی قبلی را با `hdfs dfs -rm -r -f` حذف کنید.

۸.۴. ساخت pipeline چندمرحله‌ای برای پروژه

برای پروژه‌ی اصلی، یک pipeline چندمرحله‌ای بسازید که 1 تا 5 Job را به ترتیب اجرا کند. ایده‌ی کلی این است که خروجی هر job، ورودی job بعدی شود.

نمونه‌ی منطقی مسیرها در HDFS:

```
/input/nginx_access.log


```

برای اینکه اجرای پروژه تکرارپذیر باشد، همه‌ی دستورهای داخل یک اسکریپت مثل `scripts/run_mapreduce.sh` قرار دهید. این اسکریپت باید کارهای زیر را انجام دهد:

- ساخت پوشه‌های لازم در HDFS؛
- انتقال فایل‌های ورودی از مسیر `/project/data/` به HDFS با `hdfs dfs -put`
- حذف خروجی قبلی هر job قبل از اجرای دوباره؛
- اجرای Hadoop Streaming برای هر job با mapper و reducer مربوط به همان مرحله؛
- خواندن خروجی `part-*` هر مرحله از HDFS و ذخیره نسخه‌ی نهایی لازم در مسیر `/project/outputs/`؛

- استفاده از خروجی مرحله‌ی قبلی به عنوان ورودی مرحله‌ی بعدی.

نمونه‌ی خیلی ساده از ساختار یک pipeline:

```
#!/usr/bin/env bash
set -e

cd /project

hdfs dfs -mkdir -p /input
hdfs dfs -put -f data/nginx/nginx_access.log /input/nginx_access.log

hdfs dfs -rm -r -f /output/job1
hadoop jar /opt/hadoop-3.2.1/share/hadoop/tools/lib/hadoop-streaming-3.2.1.jar \
-files mapreduce/job1_parse_clean/mapper.py,mapreduce/job1_parse_clean/reducer.py \
-mapper "python3 mapper.py" \
-reducer "python3 reducer.py" \
-input /input/nginx_access.log \
-output /output/job1

mkdir -p outputs/job1
hdfs dfs -cat /output/job1/part-* > outputs/job1/cleanednginx_logs.csv

# Then run Job 2, Job 3, Job 4, and Job 5 in the same script.
# Each job should clearly define its input path and output path.
```

در تحویل نهایی، همین اسکریپت باید همه‌ی Jobهای اجباری پروژه را اجرا کند و در پایان فایل‌های خروجی مورد انتظار، مخصوصاً outputs/final/summary.json را بسازد.

نتیجه مورد انتظار

بعد از اجرای کامل scripts/run_mapreduce.sh، باید خروجی‌های میانی در پوشه‌ی outputs/ خروجی نهایی در مسیر outputs/final/summary.json روی سیستم میزبان قابل مشاهده باشند.

۹. MapReduce Jobهای اجباری

در این فاز، pipeline پردازش باید شامل پنج MapReduce job اجباری باشد. این jobها شامل تحلیل‌های عمومی Nginx و تحلیل‌های مخصوص سرویس‌ها هستند.

۹.۱. Job 1: Parsing and Cleaning

ورودی:

```
data/nginx/nginx_access.log
data/service_logs/match_service.log
data/service_logs/team_service.log
data/service_logs/stadium_service.log
```

وظایف:

- parse کردن Nginx gateway log و لاگ‌های سرویس‌ها؛
- جدا کردن رکوردهای نامعتبر در یک فایل مستقل؛
- برای لاگ Nginx: تبدیل request_time_sec به request_time_ms؛
- برای لاگ سرویس‌ها: نگه‌داشتن فیلدهای مربوط به موجودیت entity_type و entity_value که خود سرویس تولید کرده است؛
- تولید لاگ‌های تمیزشده با schema ثابت.

نکته

«رکورد نامعتبر» یعنی خطی که JSON آن خراب است، یا یکی از فیلدهای اجباری را ندارد، یا status_code / زمان آن عددی نیست. چون لاگ‌ها ساختاریافته تولید می‌شوند، معمولاً invalid_logs.csv کوچک یا خالی است و این طبیعی است.

خروجی‌های اجباری:

```
outputs/job1/cleaned_nginx_logs.csv
outputs/job1/cleaned_service_logs.csv
outputs/job1/invalid_logs.csv
```

cleaned_nginx_logs.csv باید حداقل این فیلدها را داشته باشد:

```
timestamp,request_id,client_country,scenario,service,method,path,status_code,request_time_ms,user_agent
```

cleaned_service_logs.csv باید حداقل این فیلدها را داشته باشد:

```
timestamp,request_id,client_country,service,endpoint,entity_type,entity_value,status_code,processing_time_ms,
event_type
```

۹.۲ Job 2: General Nginx Aggregation

این job روی cleaned_nginx_logs.csv (خروجی Job 1) اجرا می‌شود و فقط تحلیل‌های عمومی و -gateway- Nginx level را تولید می‌کند.

وظایف:

- شمارش تعداد درخواست‌ها به تفکیک سرویس؛
- شمارش تعداد درخواست‌ها به تفکیک endpoint؛
- محاسبه تعداد پاسخ‌های موفق، خطاهای 4xx و خطاهای 5xx؛
- محاسبه error rate برای هر سرویس؛
- محاسبه میانگین response time برای هر سرویس و endpoint؛

- تولید آمار به تفکیک scenario.

خروجی‌های اجباری:

```
outputs/job2/service_stats.csv
outputs/job2/endpoint_stats.csv
outputs/job2/scenario_stats.csv
```

۹.۳ Job 3: Country-Entity Request Count

این job تحلیل‌های مخصوص سرویس‌ها را شروع می‌کند و ورودی آن cleaned_service_logs.csv است (لاگ سرویس‌ها، نه لاگ Nginx). هدف این job، شمارش تعداد درخواست‌های هر کشور برای هر موجودیت است. برای team-service، منظور از موجودیت، تیم است. برای match-service، منظور تاریخ بازی است. برای stadium-service، منظور ورزشگاه یا شهر است.

نمونه ورودی مفهومی:

```
country A -> request team B
country A -> request team B
country A -> request team Z
country B -> request team Z
```

خروجی مورد انتظار برای team-service:

```
country,team,total_requests
A,B,2
A,Z,1
B,Z,1
```

منطق کلی MapReduce:

```
Mapper: (country, service, entity_type, entity_value) -> 1
Reducer: (country, service, entity_type, entity_value) -> sum
```

خروجی‌های اجباری:

```
outputs/job3/country_team_requests.csv
outputs/job3/country_matchday_requests.csv
outputs/job3/country_stadium_requests.csv
```

۹.۴ Job 4: Popular Entity by Country

این job خروجی Job 3 را می‌گیرد و برای هر کشور، محبوب‌ترین موجودیت را پیدا می‌کند. برای مثال، اگر خروجی Job 3 چنین باشد:

```
country,team,total_requests
A,B,2
A,Z,1
B,Z,1
```

خروجی Job 4 باید چنین باشد:

```
country,popular_team,total_requests
A,B,2
B,Z,1
```

منطق کلی MapReduce:

```
Mapper: country -> (entity_value, total_requests)
Reducer: for each country, select entity_value with maximum total_requests
```

خروجی‌های اجباری:

```
outputs/job4/popular_team_by_country.csv
outputs/job4/popular_matchday_by_country.csv
outputs/job4/popular_stadium_by_country.csv
```

۹.۵ Job 5: Final Report Generation

این job خروجی‌های عمومی gateway-level و سرویس‌محور را ترکیب می‌کند و خروجی نهایی پروژه را می‌سازد. توجه کنید که Job 2 از لاگ Nginx و Job 3/Job 4 از لاگ‌های سرویس‌ها به دست آمده‌اند. ورودی‌ها:

- خروجی‌های Job 2 (آمار عمومی Nginx)؛
- خروجی‌های Job 3 (شمارش سرویس‌محور)؛
- خروجی‌های Job 4 (محبوب‌ترین موجودیت هر کشور).

خروجی اجباری:

```
outputs/final/summary.json
```

ساختار مورد انتظار summary.json:

```
{
  "total_requests": 100000,
  "most_requested_service": "team-service",
  "highest_error_rate_service": "stadium-service",
  "slowest_endpoint": "/api/stadiums",
  "most_popular_team_overall": "Argentina",
  "most_requested_match_day_overall": "2026-06-25",
  "most_requested_stadium_overall": "New York New Jersey Stadium",
  "popular_team_by_country": {
```

```
"Iran": "Argentina",
"Germany": "Germany"
},
"predicted_champion": "Argentina",
"predicted_final": "France vs Argentina",
"predicted_final_winner": "Argentina",
"predicted_final_stadium": "New York New Jersey Stadium"
}
```

۱۰. بخش امتیازی: Spark Structured Streaming

در این بخش، دانشجوها باید با Spark Structured Streaming لاگ‌های جدید Nginx و در صورت نیاز لاگ‌های سرویس‌ها را به صورت real-time بخوانند و چند خروجی زنده تولید کنند. حداقل خروجی‌های مورد انتظار:

- تعداد درخواست‌ها در بازه‌های زمانی کوتاه؛
- error rate زنده؛
- پرتراфик‌ترین سرویس یا endpoint در لحظه؛
- محبوب‌ترین تیم هر کشور در جریان داده‌های زنده؛
- میانگین response time در بازه‌های زمانی کوتاه.

خروجی real-time می‌تواند در terminal، یک فایل خروجی، یا یک صفحه ساده قابل مشاهده باشد.

۱۰.۱. اجرای Spark (ترجیحاً با Docker)

Spark نیز مانند Hadoop می‌تواند داخل Docker اجرا شود؛ یک ایمیج آماده‌ی Spark (شامل PySpark و spark-submit) برای این کار کافی است. اجرای محلی Spark بدون Docker هم قابل قبول است. در حالت کلی یک نسخه‌ی پایدار از Spark 3.x لازم است و برنامه را در حالت `--master local[*]` اجرا می‌کنید.

نکته

تفاوت اصلی با بخش MapReduce: در MapReduce پردازش به صورت batch روی کل لاگ انجام می‌شود، اما Spark Structured Streaming داده‌ی جدید را به صورت تدریجی و زنده پردازش می‌کند و آمارها را با رسیدن داده‌ی تازه به‌روزرسانی می‌کند.

۱۰.۲. Workflow پیشنهادی برای Spark

Spark Structured Streaming باید لاگ‌های جدید را به صورت تدریجی بخواند: برای آمار عمومی زنده از `nginx_access.log` و برای تحلیل‌های مخصوص سرویس‌ها زنده (مثل محبوب‌ترین تیم هر کشور) از لاگ‌های سرویس‌ها. ساده‌ترین روش این است که یک اسکریپت کمکی خطوط جدید لاگ را به فایل‌های کوچک‌تر تقسیم

کند و داخل پوشه‌ی ورودی Spark قرار دهد. منبع تحلیل همچنان لاگ Nginx و لاگ سرویس‌هاست، نه فایل کمکی traffic generator.

توجه

در Structured Streaming، خواندن از پوشه معمولاً برای فایل‌های جدید انجام می‌شود. بنابراین بهتر است برای تست Spark، فایل‌های کوچک جدید مثل batch_001.jsonl و batch_002.jsonl داخل پوشه ورودی ساخته شوند، نه اینکه فقط یک فایل قدیمی را append کنید.

ساختار پیشنهادی پوشه‌ها:

```
data/stream/nginx/      # input files for Spark
checkpoints/spark/      # checkpoint directory
spark/streaming_app.py
```

نمونه اجرای Spark:

```
mkdir -p data/stream/nginx checkpoints/spark

spark-submit --master local[*] \\\
    spark/streaming_app.py \\\
    --input data/stream/nginx \\\
    --checkpoint checkpoints/spark
```

توضیح دستور:

- spark-submit برنامه Spark را اجرا می‌کند.
- --master local[*] یعنی برنامه روی همین سیستم و با همه هسته‌های موجود اجرا شود.
- spark/streaming_app.py فایل اصلی برنامه Spark است.
- --input مسیر پوشه‌ای است که Spark باید فایل‌های لاگ جدید Nginx را از آن بخواند.
- --checkpoint مسیر ذخیره وضعیت streaming query است. بدون checkpoint، ادامه‌دادن یا مدیریت وضعیت stream قابل اعتماد نیست.

در یک ترمینال دیگر، ترافیک را از مسیر Nginx تولید کنید:

```
python3 traffic-generator/generate.py \\\
    --nginx-url http://localhost:8080 \\\
    --requests 1000
```

سپس خطوط جدید لاگ Nginx را به شکل فایل‌های کوچک وارد پوشه stream کنید. این کار می‌تواند با یک اسکریپت کمکی انجام شود، مثلاً:

```
python3 scripts/export_nginx_log_batches.py \
--source data/nginx/nginx_access.log \
--output data/stream/nginx \
--batch-size 200
```

برای تست دستی نیز می‌توانید چند فایل کوچک ساخته‌شده توسط خودتان را یکی‌یکی وارد پوشه stream کنید:

```
cp data/manual_stream_batches/nginx_batch_001.jsonl data/stream/nginx/batch_001.jsonl
sleep 5
cp data/manual_stream_batches/nginx_batch_002.jsonl data/stream/nginx/batch_002.jsonl
```

در پیاده‌سازی بخش امتیازی، نکات کلیدی زیر را رعایت کنید: برنامه‌ی Spark باید با readStream از پوشه‌ی ورودی (فایل‌های JSON Lines) بخواند، در پنجره‌های زمانی کوتاه (مثلاً ۱۰ ثانیه‌ای) آمار هر سرویس را جمع بزند و خروجی زنده تولید کند. استخراج موجودیت از داده، محاسبه‌ی محبوب‌ترین تیم هر کشور و سایر خروجی‌ها بر عهده‌ی خود دانشجوست.

توجه

در لاگ Nginx، مقدار request_time_sec به‌صورت رشته (string) ذخیره می‌شود. اگر آن را در Spark پردازش می‌کنید، حتماً نوع آن را به double تبدیل (cast) کنید؛ در غیر این صورت ممکن است مقدار آن null شود.

برای اجرای تمیز از ابتدا، اگر می‌خواهید وضعیت قبلی stream پاک شود، می‌توانید پوشه checkpoint را حذف کنید:

```
rm -rf checkpoints/spark
```

نتیجه مورد انتظار

در بخش امتیازی، صرفاً اجرای یک کد Spark کافی نیست. باید هنگام تولید request‌های جدید، خروجی زنده در terminal، فایل یا صفحه ساده دیده شود و دانشجو بتواند توضیح دهد که Spark چگونه داده‌های جدید لاگ‌ها را خوانده و آمارها را به‌روزرسانی کرده است.

۱۱. نحوه اجرای مورد انتظار

برای اجرای سرویس‌ها و Nginx، باید بتوان از دستور زیر استفاده کرد:

```
docker compose up --build
```

بعد از بالا آمدن سرویس‌ها، ترافیک باید با برنامه generator تولید شود. نمونه دستور:

```
python traffic-generator/generate.py --requests 100000 --nginx-url http://localhost:8080
```

پس از تولید ترافیک، فایل `nginx_access.log` باید شامل لاگ درخواست‌هایی باشد که از Nginx عبور کرده‌اند و فایل‌های `data/service_logs/*.log` نیز باید لاگ‌های اختصاصی سرویس‌ها را داشته باشند. سپس pipeline مربوط به MapReduce باید از داخل محیط Hadoop که با `docker-compose.yml` نمونه بالا آمده است اجرا شود. فایل‌های ورودی و خروجی باید از طریق مسیرهای `volume mount` شده در دسترس کانتینر باشند. نمونه نام پیشنهادی برای اسکریپت اجرا:

```
bash scripts/run_mapreduce.sh
```

نتیجه مورد انتظار

در پایان اجرای فاز اول، فایل `outputs/final/summary.json` باید ساخته شده باشد و خروجی‌های میانی هر job نیز در مسیرهای مشخص شده وجود داشته باشند.

۱۲. تست دستی و مشاهده خروجی‌ها

پس از اجرای `docker compose up --build`، می‌توانید بعضی از endpointها را مستقیماً در مرورگر باز کنید تا پاسخ JSON سرویس‌ها را ببینید. برای مثال، بسته به پیاده‌سازی خودتان، مسیرهایی شبیه موارد زیر باید پاسخ قابل مشاهده داشته باشند:

```
http://localhost:8080/api/teams?name=Argentina
http://localhost:8080/api/matches?date=2026-06-25
http://localhost:8080/api/stadiums?name=New+York+New+Jersey+Stadium
```

توجه

تست با مرورگر فقط برای دیدن پاسخ JSON مناسب است. برای تولید لاگ کامل و معنی‌دار باید headerهای `X-Request-ID`، `X-Client-Country` و `X-Scenario` ارسال شوند؛ مرورگر معمولی این headerها را به راحتی اضافه نمی‌کند. برای تست اصلی از `curl`، `Postman`، `Thunder Client` یا ابزار مشابه استفاده کنید.

نمونه تست با `curl`:

```
curl -H "X-Request-ID: manual-001" \
-H "X-Client-Country: Iran" \
-H "X-Scenario: normal" \
"http://localhost:8080/api/teams?name=Argentina"
```

برای مشاهده خروجی‌های نهایی در مرورگر، می‌توانید از ریشه پروژه یک HTTP server ساده اجرا کنید:

```
python3 -m http.server 9000
```

سپس فایل‌های خروجی را از مسیرهایی مثل موارد زیر در مرورگر ببینید:

```
http://localhost:9000/outputs/final/summary.json
http://localhost:9000/outputs/job2/service_stats.csv
http://localhost:9000/outputs/job4/popular_team_by_country.csv
```

۱۳. فایل‌های شروع کار و فایل‌های قابل تحویل

در شروع پروژه، کد پایه‌ی سه سرویس backend و یک فایل docker-compose.yml نمونه برای محیط Hadoop در اختیار دانشجویان قرار می‌گیرد. کدهای سرویس فقط پایه‌ی endpoint را فراهم می‌کنند و فایل Docker Compose نمونه نیز فقط برای اجرای محیط Hadoop Streaming است. سایر بخش‌های پروژه باید توسط گروه پیاده‌سازی یا تولید شوند.

۱۳.۱. فایل‌هایی که در اختیار دانشجویان قرار می‌گیرد

موارد زیر در اختیار دانشجویان قرار می‌گیرد:

- کد پایه‌ی match-service
- کد پایه‌ی team-service
- کد پایه‌ی stadium-service
- فایل docker-compose.yml نمونه برای بالا آوردن محیط Hadoop Streaming.

کد پایه‌ی سرویس‌ها شامل مسیرهای اصلی API و پاسخ‌های ساده است، اما بخش‌های تحلیلی آن کامل نیست. فایل Docker Compose نمونه نیز فقط برای اجرای محیط Hadoop است و شامل جواب سرویس‌ها، Nginx، traffic generator یا کدهای MapReduce نیست.

در کد پایه‌ی سرویس‌ها، بخش‌های زیر با TODO مشخص می‌شوند و باید توسط دانشجویان تکمیل شوند:

- تولید لاگ ساختاریافته‌ی سرویس‌ها؛
- استفاده از مقدارهای X-Request-ID، X-Client-Country و X-Scenario در رکورد لاگ ساختاریافته؛
- محاسبه‌ی processing_time_ms
- نوشتن فیلدهای entity_type و entity_value.

۱۳.۲. بخش‌هایی که باید توسط دانشجویها تکمیل شوند

موارد زیر بخش اصلی پروژه هستند و باید توسط گروه پیاده‌سازی یا تکمیل و در تحویل نهایی قرار داده شوند:

- تکمیل بخش‌های TODO در سرویس‌ها، مخصوصاً تولید لاگ ساختاریافته‌ی سرویس‌ها؛
- نوشتن Dockerfile هر سرویس و آماده‌کردن اجرای سرویس‌ها داخل Docker؛
- نوشتن فایل‌های nginx.conf و docker-compose.yml؛
- نوشتن برنامه traffic generator؛
- استفاده از docker-compose.yml نمونه‌ی Hadoop و هماهنگ‌کردن مسیرهای volume mount با ساختار پروژه؛
- نوشتن فایل‌های reducer, mapper و اسکریپت scripts/run_mapreduce.sh؛
- ساختن لاگ‌های اجرای نهایی و همه خروجی‌های مسیر outputs/؛
- پیاده‌سازی Spark در صورت انجام بخش امتیازی.

۱۴. موارد تحویلی فاز اول

موارد تحویلی

موارد زیر باید در تحویل فاز اول وجود داشته باشند:

- سورس‌کد تکمیل‌شده‌ی سه سرویس match-service, team-service و stadium-service، شامل بخش لاگ‌گذاری ساختاریافته؛
- Dockerfile هر سه سرویس؛
- فایل nginx.conf؛
- فایل docker-compose.yml؛
- کد Python مربوط به traffic generator؛
- استفاده از فایل docker-compose.yml نمونه‌ی Hadoop برای اجرای Hadoop Streaming و قرار دادن ورودی‌ها/خروجی‌ها در مسیرهای volume mount شده؛
- فایل nginx_access.log مربوط به اجرای نهایی؛
- فایل‌های لاگ سرویس‌ها در مسیر data/service_logs/ مربوط به اجرای نهایی؛
- کدهای mapper و reducer برای همه MapReduce job‌ها؛

- اسکریپت اجرای پشت‌سرهم MapReduce job ها؛
- همه خروجی‌های میانی داخل پوشه outputs؛
- فایل نهایی outputs/final/summary.json؛
- یک گزارش ساده PDF شامل اسکرین‌شات‌های اجرای موفق کانتینرها، تست درخواست از مسیر Nginx، بخشی از لاگ نهایی Nginx، بخشی از لاگ‌های نهایی سرویس‌ها، اجرای traffic generator، اجرای Hadoop Streaming، خروجی‌های میانی و summary.json؛
- در صورت انجام بخش امتیازی، کد Spark Structured Streaming و تصویر از خروجی real-time.

نکته

در زمان ارائه، صرفاً وجود فایل‌ها کافی نیست. پروژه باید قابل اجرا باشد و دانشجو‌ها باید بتوانند توضیح دهند که لاگ Nginx و لاگ‌های سرویس‌ها چگونه تولید شده‌اند، هر MapReduce job چه ورودی و خروجی دارد و خروجی‌های نهایی چه مفهومی دارند.